

Aplicación de Algoritmos Genéticos como Estrategia de Búsqueda Global para la Resolución del Problema de las N-Reinas

María José Zabala, Juan Andrés Bravo, David Alejandro Dávila, Arturo Manrique.

Departamento de Ingeniería de Sistemas, Pontificia Universidad Javeriana

Bogotá, Colombia

RESUMEN: *Este trabajo aborda el problema de las N-Reinas mediante un algoritmo genético. Utilizando vectores de permutación, una función de aptitud basada en colisiones y herramientas estocásticas para la diversificación, el modelo optimiza la búsqueda en espacios factoriales. Los resultados demuestran que un adecuado balance de población e iteraciones garantiza una alta eficiencia computacional.*

ABSTRACT: *This paper addresses the N-Queens problem using a genetic algorithm. Employing permutation vectors, a collision-based fitness function, and stochastic tools for diversification, the model optimizes search spaces. Results show that an optimal balance between population size and iterations ensures high computational efficiency.*

PALABRAS CLAVE: Algoritmo genético, balance poblacional, cruce PMX, mutación, optimización combinatoria, problema de las N-Reinas.

1. Introducción

El problema de las N-Reinas consiste en colocar N reinas en un tablero de ajedrez de dimensiones N x N de forma que ninguna de ellas se amenace mutuamente. En el ajedrez convencional, una reina ataca a cualquier pieza que se encuentra en su misma fila, columna o diagonal. Para modelar este problema computacionalmente, se puede emplear una representación basada en un vector de tamaño N, donde cada índice del vector representa una columna del tablero y el valor almacenado en él representa la fila en la que se ubica la reina.⁸ Por ejemplo, para un tablero de 8 reinas, considérese el vector

$$V_i = [3, 1, 6, 8, 2, 4, 7, 5], \quad \text{donde } i \in \{1, 2, \dots, 8\}$$

Siguiendo esta codificación, en la columna 1 la reina se encuentra en la fila 3, en la columna 2 en la fila 1, y así sucesivamente. (Ver Figura 1 para una representación visual de este vector en un tablero de ajedrez.)

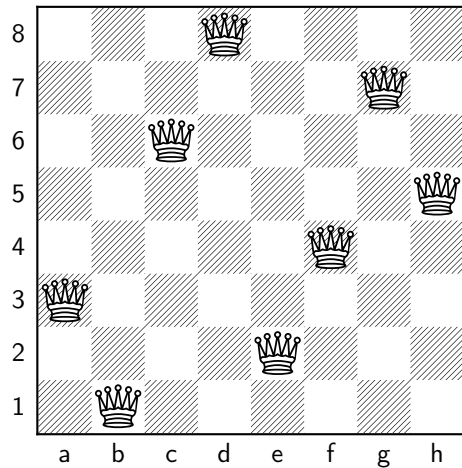


Figura 1: Configuración del tablero para $V = [3, 1, 6, 8, 2, 4, 7, 5]$ usando notación de ajedrez.

Como se puede observar, este vector contiene los números del 1 al 8 sin repeticiones, constituyendo una permutación. Gracias a este diseño, se garantiza que no existan conflictos en filas ni columnas, y el problema se reduce a evitar conflictos en las

diagonales, donde las diagonales descendientes se identifican por la ecuación $i - j = k$ y las diagonales ascendentes por $i + j = k$, siendo i y j los índices de fila y columna respectivamente, y k una constante que identifica cada diagonal.⁸

Para resolver este tipo de problemas de optimización, una primera aproximación tradicional es el uso de algoritmos de búsqueda local. Estas técnicas operan con un único estado actual, moviéndose iterativamente hacia sus vecinos. Destacan por utilizar muy poca memoria y por ser capaces de encontrar soluciones razonables en espacios grandes donde los algoritmos sistemáticos resultan ineficientes.⁶ Sin embargo, presentan limitaciones significativas, como la tendencia a quedar atrapados en óptimos locales y la dificultad de escapar de ellos.²

Al escalar el problema de un tablero fijo de 8 posiciones hacia un tamaño general de N -reinas, el espacio de búsqueda crece de manera factorial ($N!$). Aunque las técnicas de búsqueda local permiten explorar espacios grandes utilizando pocos recursos de memoria, su rendimiento se ve limitado por la presencia de óptimos locales, lo que dificulta encontrar soluciones globales.² Ante la incapacidad de la búsqueda local tradicional para garantizar una exploración global efectiva en tableros de gran tamaño, cobran relevancia los algoritmos genéticos. Inspirados en la selección natural y la evolución biológica, estos algoritmos emplean una población de soluciones candidatas que evolucionan a lo largo del tiempo mediante operadores de selección, cruce y mutación. Esta estrategia permite explorar el espacio de búsqueda de manera más amplia y efectiva, aumentando la probabilidad de encontrar soluciones óptimas o cercanas al óptimo global.⁶ En este trabajo, se implementa y evalúa un algoritmo genético para abordar el problema de las N reinas.

2. Fundamentos Teóricos

Los algoritmos genéticos (AG) son métodos de optimización global inspirados en la evolución biológica y la selección natural. A diferencia de la búsqueda local tradicional, un AG opera de forma simultánea sobre una población de soluciones candidatas y se asemeja a una variante de la búsqueda por haz estocástica, donde los nuevos estados se generan combinando características de dos padres. La calidad de cada solución se mide mediante

una función de aptitud (*fitness*), priorizando a los individuos más aptos para la reproducción.

2.1. Inspiración Biológica

Los fundamentos de la computación evolutiva se consolidaron entre las décadas de 1950 y 1970. En los años 60, investigadores como Box, Friedman, Bledsoe y Bremermann implementaron las primeras simulaciones evolutivas para optimización, seguidas por la programación evolutiva de Fogel, Owens y Walsh en 1966 basada en máquinas de estado finito. Hacia 1975, se formalizaron los sistemas adaptativos digitales mediante operadores estandarizados de selección, cruce y mutación.

2.2. Representación y Componentes

La analogía biológica se estructura de la siguiente manera:

- **Cromosoma (Individuo):** Es la codificación de una solución completa. En problemas de optimización combinatoria, existen diversas formas de representar un cromosoma, como codificación binaria, codificación de enteros, codificación de permutaciones y codificación de árboles. En el caso del problema de las N -Reinas, se utiliza una codificación de vectores de permutación de enteros.
- **Población:** Conjunto de individuos que representan soluciones candidatas al problema. La diversidad genética dentro de la población es crucial para explorar efectivamente el espacio de búsqueda y evitar la convergencia prematura hacia soluciones subóptimas. En el caso de las N -Reinas, la población inicial se genera aleatoriamente, asegurando que cada individuo sea una permutación válida de los números del 1 al N .
- **Genes y Alelos:** Cada posición del cromosoma actúa como un *gen* que especifica una característica individual, mientras que el valor asignado a ese gen se denomina *alelo* (por ejemplo, la fila específica asignada a una columna dada). En la Figura 2, se ilustra un ejemplo de cromosoma para el problema de las N -Reinas, donde se resalta un gen específico y su alelo correspondiente.³

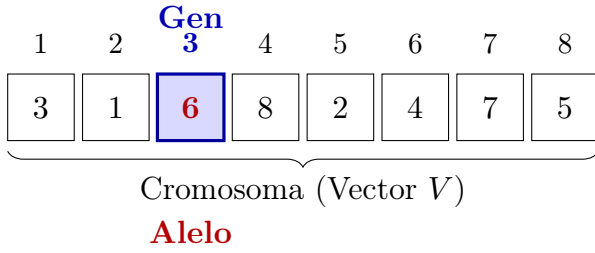


Figura 2: Representación estructural a color: el contenedor azul resalta el gen (columna) y el número rojo destaca el alelo (fila) almacenado en su interior.

2.3. Componentes Clave del Algoritmo

El proceso evolutivo se rige por la premisa de la “supervivencia del más apto”, donde las soluciones compiten por oportunidades de reproducción y transmisión genética.³ Para lograrlo, el algoritmo se compone de tres elementos fundamentales:

1. **Función de Aptitud (*Fitness*):** Es la métrica cuantitativa que evalúa qué tan adecuada es una solución candidata para resolver el problema. En el contexto de optimización, devuelve valores más altos (o penalizaciones más bajas) para los estados óptimos o más cercanos al objetivo.⁶
2. **Operadores de Selección:** Mecanismos que eligen a los individuos más aptos de la población actual para que actúen como padres en la siguiente generación, asegurando que los mejores genes tengan mayor probabilidad de heredarse.³
3. **Operadores de Recombinación y Mutación:**
 - *Cruce (Crossover):* Combina el material genético de dos padres para crear descendientes que heredan las mejores características parciales de ambos.
 - *Mutación:* Modifica aleatoriamente pequeños fragmentos del cromosoma (por ejemplo, intercambiando dos valores en el vector). Su propósito principal es introducir diversidad genética en la población y evitar el estancamiento prematuro en óptimos locales.³

3. Algoritmo y pseudocódigo

Cada individuo $V = [v_1, \dots, v_N]$ es una permutación de $\{1, \dots, N\}$. Por tanto, no existen conflictos en filas ni en columnas y solo es necesario contar los conflictos diagonales:

$$C(V) = \sum_{i=1}^{N-1} \sum_{j=i+1}^N \mathbf{1}(|v_i - v_j| = |i - j|).$$

La función de aptitud se define como $f(V) = \binom{N}{2} - C(V)$; una solución es válida cuando $C(V) = 0$. El Algoritmo 1 resume la implementación empleada: selección por truncamiento/ordenamiento sobre los mejores individuos, cruce PMX, mutación por intercambio gen a gen y elitismo de dos individuos.

Algorithm 1 Algoritmo genético implementado para las N-Reinas

Entrada: Dimensión N , población μ , generaciones $G_{\max}$, tasa de mutación p_m

Salida: Permutación V con aptitud óptima o la mejor alcanzada

```

1:  $\mathcal{P} \leftarrow \text{POBLACIÓNINICIALVALIDA}(\mu, N)$ 
2: para  $g \leftarrow 1$  hasta  $G_{\max}$  hacer
3:    $\mathcal{P} \leftarrow \text{ORDENARPORAPTITUDDESCENDENTE}(\mathcal{P})$ 
4:   si  $f(\mathcal{P}[1]) = \binom{N}{2}$  entonces
5:     devolver  $\mathcal{P}[1]$   $\triangleright$  Solución óptima sin conflictos
6:   fin si
7:    $\mathcal{P}' \leftarrow \emptyset$ 
8:   para  $i \leftarrow 1$  hasta  $\lfloor \frac{2}{3}\mu \rfloor - 1$  paso 2 hacer  $\triangleright$  Cruce de parejas elite
9:      $\mathcal{P}' \leftarrow \mathcal{P}' \cup \{\text{PMX}(\mathcal{P}[i], \mathcal{P}[i + 1]), \text{PMX}(\mathcal{P}[i + 1], \mathcal{P}[i])\}$ 
10:   fin para
11:   mientras  $|\mathcal{P}'| < \mu - 2$  hacer  $\triangleright$  Completar hijos con el top 50%
12:      $X, Y \leftarrow \text{MUESTREOALEATORIO}(\mathcal{P}[1 \dots \lfloor \mu/2 \rfloor], 2)$ 
13:      $\mathcal{P}' \leftarrow \mathcal{P}' \cup \{\text{PMX}(X, Y)\}$ 
14:   fin mientras
15:   para todo  $H \in \mathcal{P}'$  hacer  $\triangleright$  Mutación por intercambio gen a gen
16:     para  $j \leftarrow 1$  hasta  $N$  hacer
17:       si  $\text{ALEATORIO}(0, 1) < p_m$  entonces
18:          $k \leftarrow \text{ALEATORIOENTERO}(1, N)$ 
19:          $\text{INTERCAMBIAR}(H[j], H[k])$ 
20:       fin si
21:     fin para
22:   fin para
23:    $\mathcal{P} \leftarrow \mathcal{P}' \cup \{\mathcal{P}[1], \mathcal{P}[2]\}$   $\triangleright$  Elitismo (2 mejores)
24: fin para
25: devolver  $\mathcal{P}[1]$ 

```

El cruce parcialmente asignado copia en el descendiente una subcadena del primer padre y construye una correspondencia entre los alelos de esa subcadena y los del mismo intervalo en el segundo padre. Las posiciones restantes se completan aplicando dicho mapeo, lo cual evita valores duplicados y mantiene una permutación válida.¹ Su procedimiento se presenta en el Algoritmo 2.

Algorithm 2 Cruce parcialmente asignado (PMX)

Entrada: Padres X y Y , ambos permutaciones de longitud N **Salida:** Un descendiente H que también es una permutación

```
1: Elegir al azar  $a, b$  tales que  $1 \leq a < b \leq N$ 
2: Inicializar  $H$  con  $N$  posiciones vacías
3:  $H[a \dots b] \leftarrow X[a \dots b]$ 
4: para  $i \leftarrow 1$  hasta  $N$  hacer
5:   si  $i < a$  o  $i > b$  entonces
6:      $y \leftarrow Y[i]$ 
7:     mientras  $y \in X[a \dots b]$  hacer
8:        $j \leftarrow \text{POSICIÓN}(X, y)$ 
9:        $y \leftarrow Y[j]$ 
10:    fin mientras
11:     $H[i] \leftarrow y$ 
12:  fin si
13: fin para
14: devolver  $H$ 
```

4. Aplicaciones (problema de las N-Reinas)

4.1. Implementación

Representación del individuo y cromosoma.

La representación de cada individuo fue realizada mediante un arreglo de NumPy de tamaño N , donde cada índice representa una columna del tablero y el valor almacenado representa la fila en la que se ubica la reina. Siempre se garantiza que cada cromosoma sea una permutación válida de los números del 0 al $N - 1$. El estado inicial de la población se genera aleatoriamente; esta representación restringe el espacio de N^N a $N!$ al garantizar que cada cromosoma es una permutación válida.

Función de aptitud.

Para la evaluación de la aptitud, se implementó una función que calcula el número de pares de reinas que se atacan entre sí. Para esta función, en lugar de realizar comparaciones par a par $O(N^2)$, se implementó un algoritmo de conteo de colisiones diagonales en $O(N)$, utilizando dos arreglos auxiliares para contar el número de reinas en cada diagonal ascendente y descendente. La función de aptitud se define como $f(V) = \binom{N}{2} - C(V)$, donde $C(V)$ es el número de pares de reinas que se atacan entre sí.

Operador de cruce.

Se implementó el operador de cruce parcialmente asignado (PMX) para generar descendientes que son una permutación válida de los padres, evitan-

do duplicados y manteniendo la validez del cromosoma.

Selección y elitismo.

El método de selección desarrollado toma el top $2\mu/3$ de la población y realiza un cruce sistemático entre ellos, mientras que el resto de los descendientes se generan mediante muestreo aleatorio del top 50 % de la población. Se implementó elitismo de dos individuos, asegurando que los dos mejores individuos de la generación actual se mantengan en la siguiente generación.

Mutación.

El operador de mutación utilizado intercambia alelos aleatoriamente con una probabilidad p_m , donde p_m es la tasa de mutación.

4.2. Resultados y Hallazgos

Para evaluar la escalabilidad del algoritmo genético ante el crecimiento factorial del espacio de búsqueda ($N!$), se contrastaron 1 140 experimentos multidimensionales ($N \in [8, 64]$) frente a un estudio de caso en escala masiva ($N = 100$). Los resultados evidencian que el tamaño del tablero condiciona radicalmente la dinámica de convergencia, la sensibilidad de los hiperparámetros y la velocidad de resolución.

En primer lugar, la dificultad del problema se segmenta en tres categorías operativas: en tableros pequeños ($N \leq 20$), el algoritmo exhibe alta robustez con una tasa de éxito global del 81.5 % y una mediana de convergencia en apenas 12 generaciones. En tableros medianos ($21 \leq N \leq 40$), la tasa de éxito general se reduce al 27.5 % (mediana de 74 épocas), mientras que en tableros grandes ($41 \leq N \leq 64$) desciende al 12.9 % (mediana de 173 épocas). No obstante, esta disminución no refleja una incapacidad intrínseca del algoritmo: al calibrar la tasa de mutación en su franja óptima (p_m^*), se logró un 100 % de convergencia a cero colisiones en los 57 tamaños de tablero evaluados, extendiéndose exitosamente hasta $N = 100$ ($100! \approx 9,33 \times 10^{157}$) en 4 732 generaciones mediante el refuerzo poblacional ($m = 300$).

En segundo lugar, el análisis topográfico bidimensional (N vs p_m , Figura 3) revela que el incremento de N comprime severamente el corredor de mutación viable. Mientras que en tableros pequeños cualquier mutación entre 0.01 y 0.20

permite converger, para $N \geq 41$ las tasas superiores a 0.04 destruyen los bloques constructivos del cruce PMX, provocando un colapso del éxito al 0 %. A partir de este fenómeno se formuló la *Ley de Invarianza de la Perturbación Genómica*: para preservar la estabilidad del cromosoma, el número esperado de intercambios debe conservarse constante en torno a una mediana de $\mathbb{E}[\text{swaps}] = N \times p_m^*(N) \approx 0,62$, deduciendo la regla de escalabilidad $p_m^*(N) \approx 0,6/N$.

Finalmente, la dinámica temporal confirma un comportamiento bifásico universal invariante a la escala: una fase macroscópica inicial donde el cruce PMX reduce más del 80 % de las colisiones en las primeras decenas de generaciones, sucedida por una meseta asintótica donde la mutación puntual bajo elitismo resuelve los últimos conflictos diagonales.

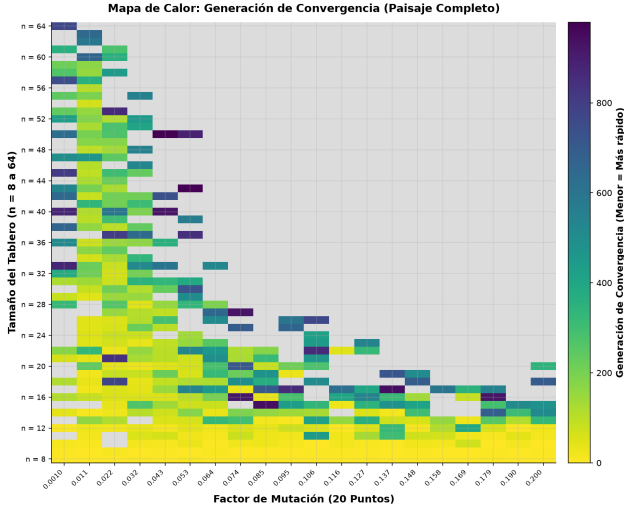


Figura 3: Mapa de calor bidimensional de convergencia (N vs p_m) ilustrando la compresión del corredor óptimo en tableros grandes.

5. Conclusiones

Los resultados obtenidos confirman la hipótesis planteada en la introducción: frente al crecimiento factorial del espacio de búsqueda del problema de las N-Reinas, un algoritmo genético con representación de permutación, cruce PMX y elitismo logra sostener una capacidad de exploración global que las técnicas de búsqueda local tradicionales no pueden garantizar. La restricción del espacio de N^N a $N!$ mediante la codificación por permutación, junto con el cálculo de aptitud en $O(N)$, resultó determinante para que la búsqueda fuera

computacionalmente viable incluso en instancias grandes.

La evidencia experimental muestra, sin embargo, que esta capacidad de exploración no es uniforme: a medida que N crece, el algoritmo con parámetros fijos ve reducida su tasa de éxito (de 81.5 % en tableros pequeños a 12.9 % en tableros grandes), lo que en un análisis superficial podría interpretarse como una limitación estructural del método. El hallazgo más relevante de este trabajo es que dicha caída no refleja una debilidad del algoritmo genético en sí, sino una mala calibración de la tasa de mutación frente a la escala del problema. Al ajustar p_m según la regla de escalabilidad derivada empíricamente ($p_m^*(N) \approx 0,6/N$), se alcanzó un 100 % de convergencia en los 57 tamaños de tablero evaluados, e incluso se resolvió exitosamente el caso $N = 100$, un espacio de búsqueda de aproximadamente $9,33 \times 10^{157}$ permutaciones. Esto evidencia que, para algoritmos genéticos con codificación de permutación, la tasa de mutación no debe tratarse como un hiperparámetro fijo, sino como una función decreciente del tamaño del cromosoma que preserva constante el número esperado de intercambios por individuo.

El comportamiento bifásico observado en la convergencia —una fase inicial rápida dominada por el cruce PMX, seguida de una fase asintótica donde la mutación puntual bajo elitismo resuelve los conflictos residuales— es consistente con el rol complementario que la teoría asigna a ambos operadores: el cruce explora combinando bloques constructivos ya buenos, mientras que la mutación introduce la diversidad necesaria para escapar de los últimos óptimos locales sin degradar el resto del cromosoma.

Como trabajo futuro, sería valioso extender el análisis a esquemas de mutación adaptativa que ajusten p_m dinámicamente durante la ejecución en lugar de fijarla a priori, así como evaluar otros operadores de cruce orientados a permutaciones (por ejemplo, OX o CX) para contrastar su efecto sobre la *Ley de Invarianza de la Perturbación Genómica* propuesta. Asimismo, resultaría interesante estudiar la escalabilidad del algoritmo más allá de $N = 100$, así como el efecto de mecanismos de diversidad poblacional (como el reinicio parcial o la migración entre subpoblaciones) para reducir aún más el número de generaciones requeridas en tableros de gran tamaño.

Referencias

- [1] D. E. Goldberg y R. Lingle, “Alleles, loci, and the traveling salesman problem”, en *Proceedings of the 1st International Conference on Genetic Algorithms and Their Applications*, J. J. Grefenstette, Ed. Lawrence Erlbaum Associates, 1985, pp. 154–159.
- [2] INAOE, *Algoritmos de Búsqueda Local*, Coordinación de Ciencias Computacionales. Disponible en: <https://ccc.inaoep.mx/~emorales/Cursos/Busqueda/node58.html>.
- [3] IONOS, *Algoritmos genéticos: definición, funcionamiento y aplicación*, Guía digital IONOS. Disponible en: <https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/algoritmos-geneticos/>.
- [4] Laboratorio de Inteligencia Artificial, *Algoritmos Genéticos*, ITAM. Disponible en: <http://laberintos.itam.mx/algoritmos-geneticos/>.
- [5] ProgramaTutos, *Problema de la Reinas / Algoritmo Genético / C / Parte 3 : Función de aptitud (Fitness)*, YouTube, 2023. [En línea]. Disponible en: <https://www.youtube.com/watch?v=CoXW27xsN0w>. [Accedido: 16 Sep. 2026].
- [6] S. Russell y P. Norvig, *Inteligencia Artificial: Un Enfoque Moderno*, Prentice Hall, 2004. Disponible en: <http://jdelagarza.fime.uanl.mx/IA/Libros/inteligencia-artificial-un-enfoque-moderno-stuart-j-russell.pdf>.
- [7] F. Sancho Caparrini, *Algoritmos Genéticos*, Departamento de Ciencias de la Computación e Inteligencia Artificial, Universidad de Sevilla. Disponible en: https://www.cs.us.es/~fsancho/Blog/posts/Algoritmos_Geneticos.md.html.
- [8] Wikipedia, *Problema de las ocho reinas*, Disponible en: https://es.wikipedia.org/wiki/Problema_de_las_ocho_reinas.